

MODULE 1: ADVANCED JAVA / J2EE FUNDAMENTALS

► Java Data Types & Operators

Primitive Type	Size	Default	Range / Notes
byte	8 bits	0	-128 to 127
short	16 bits	0	-32768 to 32767
int	32 bits	0	-2^{31} to $2^{31}-1$
long	64 bits	0L	-2^{63} to $2^{63}-1$
float	32 bits	0.0f	IEEE 754 single precision
double	64 bits	0.0d	IEEE 754 double precision
char	16 bits	'\u0000'	Unicode character
boolean	1 bit	false	true or false

► OOP in Java — Four Pillars

Pillar	Meaning	Java Keyword/Feature
Encapsulation	Wrapping data + methods; restrict access	private, getters/setters
Abstraction	Hiding implementation, showing interface	abstract class, interface
Inheritance	Child class inherits parent class features	extends (class), implements (interface)
Polymorphism	One name, many forms	Overloading (compile), Overriding (runtime)

★ Exam Tip: Polymorphism = method overloading (compile-time) + method overriding (runtime).
Interface supports multiple inheritance in Java.

► Exception Handling

Keyword	Purpose
try	Block of code to monitor for exceptions
catch	Handles the exception; catches specific exception type
finally	Always executes (cleanup code, close resources)
throw	Explicitly throw an exception object

throws	Declares that method may throw exception (in signature)
---------------	---

throw vs throws	throw: used inside method body to throw exception. throws: used in method declaration to declare possible exceptions.
------------------------	---

► Thread Lifecycle

State	Description
NEW	Thread created but start() not called
RUNNABLE	start() called; waiting for CPU or running
BLOCKED	Waiting for monitor lock (synchronized block)
WAITING	wait() / join() / park() — indefinitely waiting
TIMED_WAITING	sleep(ms) / wait(ms) — waiting for specified time
TERMINATED	run() method completed or exception occurred

► Collections Framework

Interface	Impl. Classes	Ordered?	Duplicates?	Key Feature
List	ArrayList, LinkedList, Vector	✓ Yes	✓ Yes	Index-based access
Set	HashSet, LinkedHashSet, TreeSet	✗ No*	✗ No	Unique elements
Queue	LinkedList, PriorityQueue, Deque	✓ FIFO	✓ Yes	FIFO processing
Map	HashMap, TreeMap, LinkedHashMap	✗ No*	Keys: No	Key-Value pairs

★ TreeSet & TreeMap maintain sorted order. LinkedHashSet & LinkedHashMap maintain insertion order.

► J2EE — Servlet Lifecycle

Step	Method	Called When
1. Load & Instantiate	Constructor	Container loads the servlet class
2. Initialize	init()	Once, after instantiation
3. Service	service() → doGet() / doPost()	Every client request
4. Destroy	destroy()	Container shuts down or reloads

► JSP — Implicit Objects & Directives

Implicit Object	Type	Use
request	HttpServletRequest	Get request parameters
response	HttpServletResponse	Send response to client
session	HttpSession	Maintain session state
application	ServletContext	App-wide attributes
out	JspWriter	Write output to client
pageContext	PageContext	Access all other implicit objects

► Spring & Hibernate — Quick Reference

Framework	Core Concept	Key Feature
Spring Core	IoC / Dependency Injection	Objects managed by Spring Container (ApplicationContext)
Spring Boot	Auto-configuration	Convention over configuration; embedded server
Hibernate	ORM (Object-Relational Mapping)	Maps Java classes to DB tables; HQL queries

MODULE 2: OOP CORE PRINCIPLES

► SOLID Principles

Letter	Principle	Meaning
S	Single Responsibility	A class should have only one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subclass must replace base class without breaking code
I	Interface Segregation	Don't force classes to implement unused methods
D	Dependency Inversion	Depend on abstractions, not concrete implementations

► Types of Inheritance

Type	Structure	Java Support
Single	$A \rightarrow B$	✓ Yes (extends)
Multilevel	$A \rightarrow B \rightarrow C$	✓ Yes
Hierarchical	$A \rightarrow B, A \rightarrow C$	✓ Yes
Multiple	$A + B \rightarrow C$	✗ No (use interfaces)
Hybrid	Mix of above	✗ Not directly (interfaces only)

► Association, Aggregation, Composition

Relationship	Dependency	Lifecycle	Example
Association	Weak — uses	Independent	Teacher — Student
Aggregation	Medium — has-a	Child survives parent	Department — Employee
Composition	Strong — part-of	Child dies with parent	House — Room

★ Composition is the strongest relationship — child cannot exist without parent (House-Room).
Aggregation allows child to exist independently (Department-Employee).

► Overloading vs Overriding

Feature	Method Overloading	Method Overriding
Type	Compile-time (Static)	Runtime (Dynamic)
Class	Same class	Parent-child class

Signature	Different parameters	Same signature
Return type	Can differ	Must be same (or covariant)
@Override	Not used	Must be used

MODULE 3: PYTHON PROGRAMMING

► Python Data Structures Comparison

Structure	Syntax	Ordered	Mutable	Duplicates	Key Use
List	[]	✓ Yes	✓ Yes	✓ Yes	General sequence
Tuple	()	✓ Yes	✗ No	✓ Yes	Fixed data, dict key
Set	{ }	✗ No	✓ Yes	✗ No	Unique elements
Dictionary	{k:v}	✓ (Py3.7+)	✓ Yes	Keys: No	Key-value mapping

★ Tuples are immutable → faster than lists, can be used as dictionary keys. Frozen sets are immutable sets.

► Python OOP vs Java OOP

Feature	Python	Java
Access Modifiers	Convention: _ (protected), __ (private)	Strict: public/private/protected
Multiple Inheritance	✓ Yes (MRO via C3)	✗ Only via interfaces
Method Overloading	✗ Not directly (use *args)	✓ Yes (different params)
Constructor	__init__(self)	ClassName()
Destructor	__del__(self)	finalize() or AutoCloseable

► Python Libraries Overview

Library	Purpose	Key Functions
NumPy	Numerical computing, arrays	np.array(), np.zeros(), np.dot(), np.reshape()
Pandas	Data manipulation, DataFrames	pd.DataFrame(), read_csv(), groupby(), merge()
os / sys	OS interaction, scripting	os.listdir(), os.path, sys.argv

MODULE 4: WEB TECHNOLOGIES

► HTML vs XML

Feature	HTML	XML
Purpose	Displaying data	Storing & transporting data
Tag definition	Predefined tags	User-defined tags
Case sensitive	✗ No	✓ Yes
Root element	<html>	Required (any name)
Validation	DTD / None	DTD / XML Schema (XSD)

► CSS Box Model

Layer	Description
Content	The actual text/image content area
Padding	Space INSIDE border (between content and border)
Border	The visible boundary around padding
Margin	Space OUTSIDE border (between elements)

★ box-sizing: border-box includes padding & border in the total width/height (preferred in modern CSS).

► JavaScript — ES6 Key Features

Feature	Old Way	ES6+ Way
Variables	var (function scoped)	let / const (block scoped)
Functions	function() {}	() => {} (Arrow function)
String	"Hello " + name	`Hello \${name}` (Template literals)
Async	Callbacks	Promises / async-await
Classes	prototype-based	class keyword

► HTTP Methods — REST

Method	Operation	Request Body	Idempotent
GET	Read / Retrieve	No	✓ Yes

POST	Create	Yes	✗ No
PUT	Update (Full replace)	Yes	✓ Yes
PATCH	Update (Partial)	Yes	✗ No
DELETE	Remove	No	✓ Yes

► HTTP Status Codes

Range	Category	Common Codes
1xx	Informational	100 Continue
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

MODULE 5: DATABASE PROGRAMMING — SQL & PL/SQL

► SQL Command Categories

Category	Commands	Description
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME	Define/modify database structure
DML	INSERT, UPDATE, DELETE, MERGE	Manipulate data in tables
DQL	SELECT	Query/retrieve data
DCL	GRANT, REVOKE	Control access/permissions
TCL	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

► SQL Joins

Join Type	Returns	Null Rows?
INNER JOIN	Matching rows from BOTH tables	No nulls
LEFT OUTER JOIN	All from left + matching from right	Right side nulls if no match
RIGHT OUTER JOIN	All from right + matching from left	Left side nulls if no match
FULL OUTER JOIN	All rows from BOTH tables	Nulls where no match
SELF JOIN	Table joined with itself	Uses aliases
CROSS JOIN	Cartesian product of both tables	All combinations

► PL/SQL Block Structure

Section	Keyword	Purpose	Mandatory?
Declaration	DECLARE	Declare variables, cursors, constants	Optional
Execution	BEGIN	Main logic, SQL statements, loops	✓ Required
Exception	EXCEPTION	Handle runtime errors	Optional
End	END;	Terminate the block	✓ Required

► Stored Procedure vs Function vs Trigger

Feature	Stored Procedure	Function	Trigger
---------	------------------	----------	---------

Returns value	Optional (OUT param)	✓ Always	✗ No
Called by	EXEC / CALL	SELECT / PL/SQL	Auto (DML events)
DML inside	✓ Yes	Restricted	✓ Yes
Use case	Business logic	Calculation/value	Auto-action on event

MODULE 6: UNIX / LINUX SHELL PROGRAMMING

► Essential Linux Commands

Command	Purpose	Example
ls	List files/directories	ls -la (detailed list)
cp	Copy files	cp src.txt dest.txt
mv	Move/rename files	mv old.txt new.txt
rm	Remove files	rm -rf dir/ (recursive force)
grep	Search text in files	grep -r 'error' /logs/
chmod	Change file permissions	chmod 755 script.sh
ps	List processes	ps aux (all processes)
find	Find files	find / -name '*.txt'

► chmod Permission Numbers

Number	Permission	Binary
7	Read + Write + Execute (rwx)	111
6	Read + Write (rw-)	110
5	Read + Execute (r-x)	101
4	Read only (r--)	100
0	No permissions (---)	000

★ **chmod 755** = Owner:rwx, Group:r-x, Others:r-x. **chmod 644** = Owner:rw-, Group:r--, Others:r-- (common for web files)

► Shell Scripting Essentials

Concept	Syntax	Example
Shebang	<code>#!/bin/bash</code>	First line of every script
Variable	<code>NAME=value</code>	<code>NAME="World"; echo \$NAME</code>
If-else	<code>if [cond]; then...fi</code>	<code>if [\$x -gt 10]; then echo "big"; fi</code>
For loop	<code>for i in list; do...done</code>	<code>for i in {1..5}; do echo \$i; done</code>
While loop	<code>while [cond]; do...done</code>	<code>while [\$n -gt 0]; do...; done</code>
Pipe	<code>cmd1 cmd2</code>	<code>ls -la grep .txt</code>

Redirect >	cmd > file	echo hello > file.txt (overwrite)
Append >>	cmd >> file	echo world >> file.txt (append)

MODULE 7: REACT FRAMEWORK & API CONCEPTS

► React — Core Concepts

Concept	Description
Component	Reusable UI piece — Functional (hooks) or Class-based
JSX	JavaScript + XML syntax — looks like HTML inside JS
Props	Read-only data passed from parent to child component
State	Mutable data local to a component (triggers re-render on change)
Virtual DOM	React's in-memory DOM; diffs with real DOM for efficient updates

► React Hooks

Hook	Purpose	Example Use
useState	Add state to functional components	<code>const [count, setCount] = useState(0)</code>
useEffect	Side effects (API calls, subscriptions)	Fetch data on component mount
useContext	Access global context (avoid prop drilling)	Theme, Auth context
useRef	Access DOM elements directly	Input focus, previous value
useReducer	Complex state management (like Redux)	Counter with actions

► Functional vs Class Components

Feature	Functional Component	Class Component
Syntax	<code>const Comp = () => {}</code>	<code>class Comp extends React.Component</code>
State	useState Hook	<code>this.state = {}</code>
Lifecycle	useEffect Hook	<code>componentDidMount</code> , etc.
Recommended	✓ Yes (modern)	Legacy (still works)

► REST vs SOAP

Feature	REST	SOAP
Protocol	Architectural style	Protocol (strict spec)
Format	JSON, XML, HTML	XML only
Speed	Fast (lightweight)	Slower (verbose XML)
Security	HTTPS, JWT, OAuth	WS-Security (built-in)
Use case	Web APIs, mobile apps	Enterprise banking, finance

MODULE 8: MOBILE DEVELOPMENT & APP SERVERS

► Android Activity Lifecycle

Method	When Called	Use
onCreate()	Activity first created	Initialize UI, set layout
onStart()	Activity visible to user	Prepare resources
onResume()	Activity in foreground, interactive	Start animations, sensors
onPause()	Partially obscured (another activity)	Save data, pause animations
onStop()	Activity not visible	Release heavy resources
onDestroy()	Activity being destroyed	Final cleanup

► Android Storage Options

Storage Type	Use When	Size Limit
SharedPreferences	Small key-value pairs (settings, prefs)	Small (a few KB)
Internal Storage	Private files, app-specific data	Device storage
External Storage	Shareable files (media, docs)	SD card / shared storage
SQLite Database	Structured relational data	Practical ~2GB
Network Storage	Cloud/remote data via API	Unlimited (server-side)

► Application Server Comparison

Server	Type	Language	Best For
Apache Tomcat	Servlet/JSP Container	Java	Java web apps, Servlets, JSP
IIS	Web Server	.NET, ASP	Microsoft/.NET applications
Node.js	Runtime Environment	JavaScript	Real-time, event-driven apps
Apache HTTP	Web Server	PHP, Static	Static files, PHP apps, reverse proxy

MODULE 9: QUICK REVISION — KEY TERMINOLOGIES

► MVC Architecture

Layer	Role	In Java Web	In React
Model (M)	Data & business logic	JavaBeans, DAO, Hibernate	State / API data
View (V)	User interface	JSP, HTML templates	Components / JSX
Controller (C)	Handles requests, connects M+V	Servlet / Spring Controller	Event handlers / Router

► JWT vs Session Authentication

Feature	JWT (Token-based)	Session-based
Storage	Client-side (localStorage / cookie)	Server-side (session store)
Scalability	✓ Stateless, scales easily	Harder (sticky sessions)
Revocation	Hard (no server state)	Easy (destroy session)
Use case	REST APIs, microservices	Traditional web apps

► Commonly Tested Terminologies

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — DB transaction properties
CORS	Cross-Origin Resource Sharing — allows/denies cross-domain requests
CRUD	Create, Read, Update, Delete — basic DB/API operations
ORM	Object-Relational Mapping — maps OOP classes to DB tables (Hibernate)
API Gateway	Single entry point for all API calls; handles routing, auth, rate limiting
Microservices	Architecture where app = many small independent services
CI/CD	Continuous Integration / Delivery — automated build+test+deploy pipeline
Idempotent	Same request = same result every time (GET, PUT, DELETE)
Cursor (PL/SQL)	Pointer to result set of a SQL query; IMPLICIT (auto) or EXPLICIT (manual)
DTD / XSD	Document Type Definition / XML Schema — define XML structure rules

★ EXAM FOCUS: OOP principles (very important ★), SQL Joins & queries, HTTP methods & REST, Thread lifecycle, Collections Framework differences.

★ Practice writing SQL JOIN queries, Java threading examples, and shell scripts for maximum score.